

Python OOP : Classes and Instances

'is a blue print' so we don't have to do this manually each time from scratch

Why classes are important:

They allow us to logically group our data and functions in a way that's easy to [reuse] and also easy to build upon if need be

Class Employee:

Pass

emp-1 = Employee()

emp-2 = Employee()

print(emp-1)

print(emp-2)

Both of these are employee objects, they both are unique they both have different locations in memory.

Note

⇒ IF we only create class name and then leave it, it will give us error: expected indented block after class definition.

⇒ Difference in class with pass and without pass statement

Class Employee:

Pass

→ This means that class is technically empty, it does not have any methods or attribute to define. You could add them later.

class employee:

→ It will not execute anything unless you add some methods or functions in it.

mandatory

attribute → state of object/instance
methods → behaviour of class.

Class is like blueprint that defines the structure and behaviour while **instance** is actual object created based that blueprint.

Instance Variable: contain data that is unique to each instance.

Creating `--init--` Method:

This is used to initialize or constructor for our class.

So, we don't have to do anything manually.

→ They receive instance as first argument automatically.

self → instance.

↓
class w/ all methods ko obj ko

Class Employee:

def `--init--`(self, first, last, pay):

self.first = first

self.last = last

self.pay = pay

self.email = first + '.' + last + '@company.com'

اگر یہاں self نہیں لکھیں تو error آئے گا

def full_name(self):

name = self.first + self.last

return name

⇒ By doing this we are going to make it instance variables.

def main():

emp-1 = Employee('Mahnoor', 'Arif', 10,000)

print(emp-1.full_name())

print(emp-1.email) → main thing.

Employee.full_name(emp-1)

main()

Class Variables: variables that are share among all instances.

Before init method. $\text{raised_amount} = 1.04$

```
def apply_raise(self):  
    self.raised_amount = Employee.raised_amount * 1.04  
    self.x = int(self.pay * 1.04)  $\Rightarrow 4. \uparrow$   
    return self.x
```

```
def main():  
    print(emp-1.pay)  
    emp-1.apply_raise()  
    print(emp-1.x)  
main()
```

Class Methods & Static Method:

$\Rightarrow$ Decorators are the functions which help to change and return other functions.

$\Rightarrow$ Regular methods in a class automatically take instance as the first argument (self).

@classmethod

def set_raised_amt(cls, amount):

By convention it take cls as first argument like self in regular class method.

(pass)

We are working with class instead of instance.

cls.raised_amt = amount

```
def main():  
    print(Employee.raised_amt)  
    print(emp-1.raised_amt)  
main()
```

Employee.set_raised_amt(1.05)

$\Rightarrow$

1.04	1.05
1.04	1.05

Class Employee:

→ class variable, we can't call class variable directly, so we call method below.

```
def __init__(self, first, last, pay):
```

```
    self.first = first
```

```
    self.last = last
```

```
    self.pay = pay
```

```
    self.email = first + last + '@company.com'
```

```
def fullname(self):
```

```
    self.name = self.first + self.last + 'name'
```

```
    return self.name
```

```
def apply_raise(self):
```

```
    self.pay = int(self.pay * self.raise_amt)
```

```
    return self.pay
```

@class method

```
def set_raise_amt(cls, amount):
```

```
    selfcls.raise_amt = amount
```

```
def main():
```

```
    emp-1 = Employee('Mahnoor', 'Arif', 50000)
```

```
    print(f'Name: {emp-1.fullname()} EMAIL: {emp-1.email}')
```

```
    Employee.set_raise_amt(1.05) → this is method we call here
```

```
    print(Employeeemp-1.raise_amt) → 1.05
```

```
main()
```

@y.setter
@property
def g

(x)

@setter
@getter

getx()

setx()

OOP: Class Method & Static Method

- ⇒ Regular method automatically pass the instance (self) and class method automatically pass the first argument (cls) but static method does not pass anything automatically.
- ⇒ Static method just behave like regular functions. We include them in our classes because they have some logical connection with the class.

Example: We add simple function of date which return it is a workday or not in employee class. This function doesn't depend on any specific instance or class variable.

@staticmethod

```
def is-workday (day):  
    if day.weekday() == 5 or day.weekday() == 6:  
        return False  
    return True
```

def main():

```
    import datetime  
    my_date = datetime.datetime(2023, 10, 10)  
    print(employee.is-workday(my_date))
```

main

output ⇒ False

Decorators:

are the functions which help to change and return other functions.

Why we use: Making our code more modular, reusable and maintainable. These are the fundamental tools for building robust and flexible software system.

output

Good Morning
Hello World
Thank for using this
function.

```
def greet (fx):  
    def mfx():  
        print('Good Morning')  
        fx()  
        print('Thank for using this function')  
    return mfx
```

```
* @greet  
def hello():  
    print('Hello world')
```

```
* @hello()  
greet(hello)() → give same output
```

Getter and Setter:

getter and setters helps us to use function's return value as an object's property. You can also set it.

Private Data Members:

are the attributes within class that are intentionally made inaccessible from outside the class. To access them, sometime we use public methods and sometime we use property and getter; setter.

@property ⇒ Declare the method as property.

Class Student:

```
def __init__(self, name):  
    self.__name = name
```

@property

```
def name(self):  
    return self.__name
```

```
s = Student('Steve')  
print(s.name)
```


→ In Previous Example, if you don't use @property then we can not access the private data members. By using @property, this behaves like public members.

→ Now, we can not modify the name, if we want to print 'Jim' instead of 'Steve', so we use the getter and setter. @name.setter

```
@name.setter
def name (self, value):
    self.__name = value
def main():
    s = student('Steve')
    print(s.name)
    s.name = 'Bill'
    print(s.name) → Bill
```

(Magic / Dunder) Methods:

They have double underscore at the beginning and end of their name. ~~These~~^{str} method provides a string representation of an obj. When you pass an object to these function, they implicitly call these function and give the output as mentioned in this method. They come and set all of our attributes for us.

→ We can't call magic method. In general, we can call but we can not.

Inheritance:

When a class derives from another class. The child class will inherit all the public and protected properties and method from parent class. In addition, it can have its own properties and method.

Syntax:

class Base class:

Body of class

class Derived class (Base class):

Body of derived class

Class Employee: id

def __init__(self, name, id):

self.name = name

self.id = id

def showDetails (self):

print (f' The name of Employees {self.id} is {self.name}')

def main(): ~~cls~~ ^{self.name} ~~cls~~ ^{cls.id}

e1 = Employee ('Rohan', 400)

e1.showDetails()

e2 = Employee ('Harry', 410)

e2.showDetails()

main()

Employee.id(e)

e1.name

⇒ if someone ask us to change the class name, if we did this, lot of error will occurs.

⇒ Agr hum asko rename krn gy tu b. problem a jaye ge

```
Class Programmer (Employee):  
    def showlanguage (self):  
        print ("The default language is python.")
```

```
def main():  
    e2 = showlanguage ()
```

by defining employee class it will give you error.

⇒ Hum sirf wohi method use kr sakte hai jo class mn define hai.

⇒ Agr hum e2 = Programmer ('Harry', 410) krn gy tu error nai aye ga.

⇒ super() function is used to give access to methods and properties of parent or sibling class.

```
Class Employee():
```

Body of code

```
Class Developer (Employee):
```

```
    def __init__(self, first, last, pay, prog-lang):
```

```
        super().__init__(first, last, pay)
```

```
    @Employee.__init__(self, first, last, pay)
```

⇒ if we want to access same arguments and add one use above syntaxes.


```
class Manager (Employee):  
    def __init__ (self, first, last, pay, employees=None):  
        super(). __init__ (first, last, pay)  
        if employees is None:
```

```
            self.employees = []
```

```
        else:
```

```
            self.employees = employees
```

```
    def add_emp (self, emp):  
        if emp not in self.employees:  
            self.employees.append(emp)
```

```
    def remove (self, emp):  
        if emp in self.employees:  
            self.employees.remove(emp)
```

```
    def print_emps (self):  
        for emp in self.employees:  
            print ('-->', emp.fullname())
```

```
def main():
```

```
    dev_1 = Developer ('Corey', 'Schaefer', 5000, 'Python')  
    mgr_1 = Manager ('Lee', 'Smith', 9000, [dev_1])
```

```
main()
```


Encapsulation: ✓

Means containing all important info inside a object and only exposing selected info to outside world. In terms of OOP, this mean keeping all field private and providing public methods for accessing and setting this data if it is needed outside the class.

Class Car :

```
def __init__(self, make, model):  
    self.__make = make  
    self.__model = model
```

```
def set make (self, make):
```

```
    self.__make = make
```

```
def get make (self):
```

```
    return  
    self.__make
```

- Make and models are private.
- Getter method allow external code to access object data.
- Setter method allow external code to modify data.

Abstraction: ✓

core concept of oop in which we handle complexity by hiding unnecessary information from the user.

- enable user to implement even more complex logic ~~on~~ without understanding or even thinking about all hidden background / back-end complexity.